

LEVEL 2 PROJECT REPORT

Static Website Hosting on AWS EC2 with Nginx

Manual Deployment of a Static HTML/CSS/JS Website on Amazon Linux Using Nginx

Technology Stack

AWS EC2 • Amazon Linux 2023 • Nginx • WinSCP (SFTP) • EC2 Instance Connect

Technology / Service	Role in Project
AWS EC2	Hosts the Amazon Linux instance running the Nginx web server.
Amazon Linux 2023	Provides the Linux environment used for installation and configuration.
Nginx	Serves the deployed HTML/CSS/JavaScript website over HTTP.
EC2 Instance Connect	Provides browser-based SSH access to the EC2 instance.
WinSCP (SFTP)	Transfers website files from the local machine to the Nginx web root.

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Prepared for Manimaran

Implementation evidence: 12 screenshots + 1 system architecture diagram

Table of Contents

1. Project Overview
 2. Problem Statement
 3. Objectives
 4. Technologies and Tools
 5. Prerequisites
 6. System Architecture
 7. Deployment Workflow
 8. Project Components
 9. Implementation Procedure
 10. Testing and Validation
 11. Troubleshooting and Operational Notes
 12. Security Considerations
 13. Advantages
 14. Limitations
 15. Future Enhancements
 16. Key Lessons Learned
 17. Interview / Viva Explanation
 18. Conclusion
- Appendix A — Implementation Evidence Inventory

1. Project Overview

This Level 2 project demonstrates the manual deployment of a static website on AWS EC2 using Nginx as the web server. An Amazon Linux 2023 EC2 instance was launched, updated, and configured with Nginx. The default Nginx web root was verified, cleaned up, and replaced with the custom website content (a portfolio-style HTML/CSS/JS site named "Mustacchio") transferred using WinSCP over SFTP. Nginx was configured to start on boot, and the deployed website was validated directly through a browser using the instance's public IP address.

The submitted evidence covers instance verification, remote connection setup, Nginx installation, service start and boot-time enablement, file transfer using WinSCP, web-directory permission correction, and browser-level validation of the final deployed site, including one instance where the connection was refused while the service was temporarily stopped.

2. Problem Statement

Hosting a static website requires provisioning a server, installing and configuring a web server, transferring the website files to the correct web root, setting correct file permissions, and validating that the site is reachable over HTTP. Performing each of these steps manually and correctly is a fundamental skill before introducing automation. This project addresses that requirement using an AWS EC2 instance running Amazon Linux, Nginx as the web server, and WinSCP for secure file transfer.

3. Objectives

- Launch and verify an AWS EC2 instance in the Mumbai (ap-south-1) region.
- Connect to the instance securely using EC2 Instance Connect (SSH).
- Update the Amazon Linux 2023 environment and install Nginx.
- Start the Nginx service and verify it is active and running.
- Enable Nginx so that it starts automatically on system boot.
- Validate the default Nginx welcome page over HTTP.
- Transfer custom website files (HTML/CSS/JS) to the Nginx web root using WinSCP.
- Correct file and directory permissions for the web content.
- Validate the deployed website through a browser using the public IP address.

4. Technologies and Tools

Technology / Service	Purpose	Role in Project
AWS EC2	Compute	Hosts the Amazon Linux instance running the Nginx web server.
Amazon Linux 2023	Operating system	Provides the Linux environment used for installation and configuration.
Nginx	Web server	Serves the deployed HTML/CSS/JavaScript website over HTTP.
EC2 Instance Connect	Secure remote access	Provides browser-based SSH access to the EC2 instance.
WinSCP (SFTP)	File transfer	Transfers website files from the local machine to the Nginx web root.

Technology / Service	Purpose	Role in Project
AWS Internet Gateway	Network access	Routes HTTP requests from the user's browser to the EC2 instance.
Security Group	Network access control	Controls inbound access to SSH (22) and HTTP on the instance.

5. Prerequisites

- An AWS account with access to EC2 and the required security-group configuration.
- An EC2 instance (Amazon Linux 2023, t3.micro) with a public IPv4 address.
- A security group allowing inbound SSH (port 22) and HTTP access.
- EC2 Instance Connect access, or an SSH key pair, for remote administration.
- WinSCP (or an equivalent SFTP client) installed on the local machine.
- The static website source files (HTML, CSS, JS, images, fonts) ready for upload.

6. System Architecture

The architecture is a single-tier web hosting setup. The user's browser sends an HTTP request that reaches AWS through the Internet Gateway. The request is routed to the EC2 instance running Amazon Linux, where Nginx listens for incoming HTTP traffic. Nginx serves the requested static website files (HTML/CSS/JS) directly from its web root back to the user's browser.

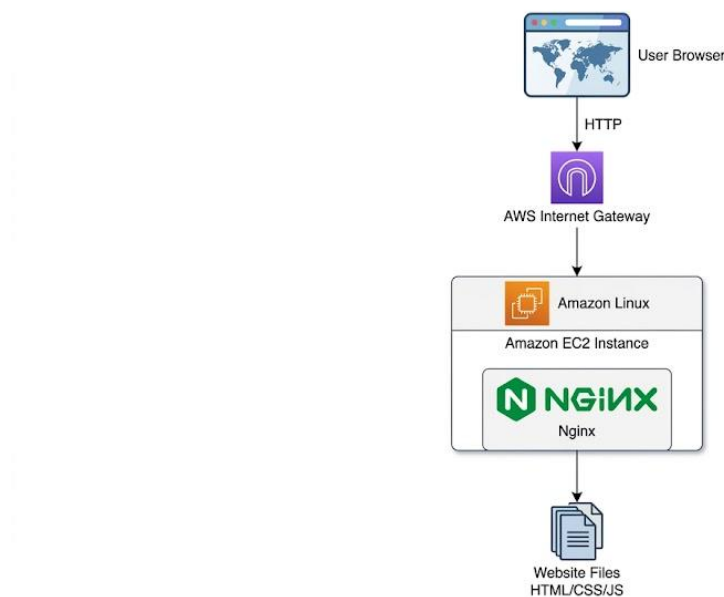


Figure 1. Level 2 static website hosting architecture — Browser → AWS Internet Gateway → EC2 (Amazon Linux + Nginx) → Website Files.

6.1 Architecture Flow

Step	Flow	Purpose
1	User Browser → AWS Internet Gateway	The browser sends an HTTP request to the public IP address of the EC2 instance.
2	Internet Gateway → EC2 Instance	AWS routes the incoming request to the Amazon Linux EC2 instance.
3	EC2 Instance → Nginx	Nginx, running on the instance, receives and processes the HTTP request.
4	Nginx → Website Files	Nginx reads the requested HTML/CSS/JS files from its web root directory.
5	Nginx → User Browser	Nginx returns the requested website content, which the browser renders.

7. Deployment Workflow

1. Launch and verify the EC2 instance in the AWS Console.
2. Connect to the instance using EC2 Instance Connect.
3. Update the Amazon Linux system packages.
4. Install Nginx using the package manager.
5. Start the Nginx service and confirm it is active.
6. Enable Nginx so it restarts automatically after a reboot.
7. Validate the default Nginx page over HTTP in a browser.
8. Transfer the custom website files to /usr/share/nginx/html/ using WinSCP.
9. Correct ownership/permission issues on the web-content directory.
10. Restart Nginx after the content update and re-validate.
11. Confirm the final deployed website renders correctly in the browser.

8. Project Components

- EC2 Instance (linux-ws): The Amazon Linux 2023, t3.micro instance that hosts the Nginx web server.
- Security Group: Controls inbound access; the evidence confirms port 22 (SSH) is open for remote administration.
- Nginx Web Server: Installed via the Amazon Linux package manager and configured to serve static content.
- Nginx Web Root (/usr/share/nginx/html/): The directory that holds the deployed website files.
- WinSCP Client: Used to transfer the local website folder ("mustacheenthusiast") to the EC2 web root over SFTP.
- Mustacchio Website: The deployed static site containing HTML pages (index, about, gallery, blog, contact), CSS, JS, fonts, and image assets.

9. Implementation Procedure

The following 12 evidence steps reproduce the supplied screenshots in chronological order. Each step places the instruction/action before its corresponding evidence image.

Step 1 — Verify the EC2 instance

Instruction / Action: The AWS EC2 console shows the instance "linux-ws" (i-007f4adcb498cc514) in the Mumbai region, of type t3.micro, in the Running state with public IP 52.66.240.244 and private IP 172.31.4.113.

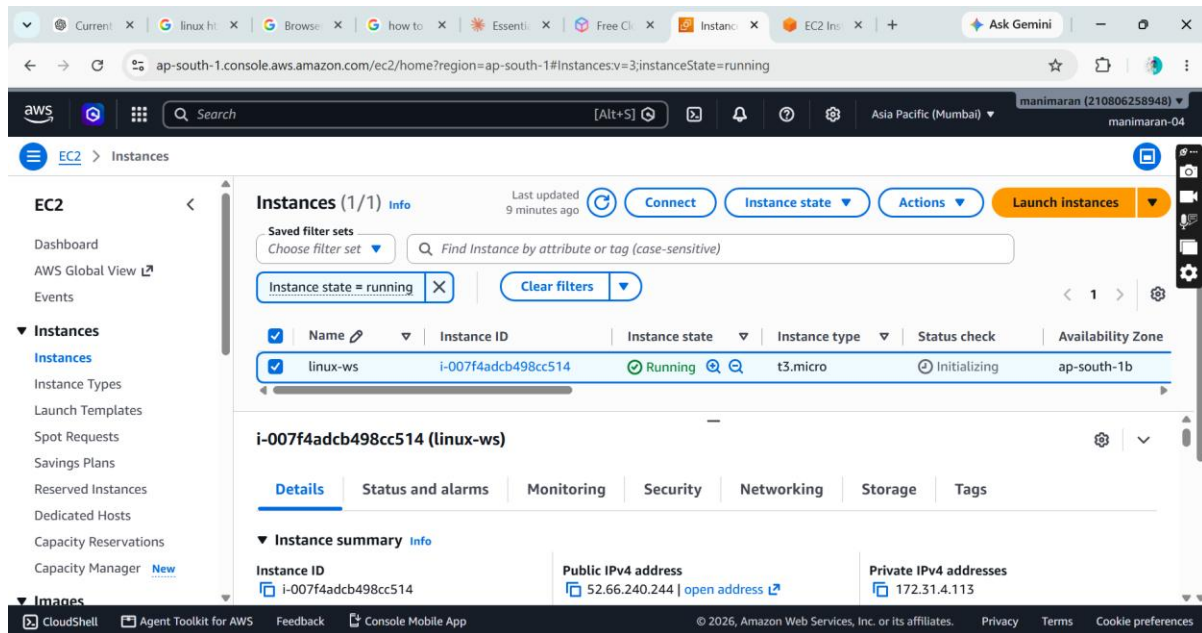


Figure 2. Evidence screenshot — Verify the EC2 instance is running.

Step 2 — Connect to the instance

Instruction / Action: The EC2 "Connect to instance" screen shows the public IPv4 address, the default username ec2-user, and a notice that port 22 (SSH) is open to all IPv4 addresses in the security group.

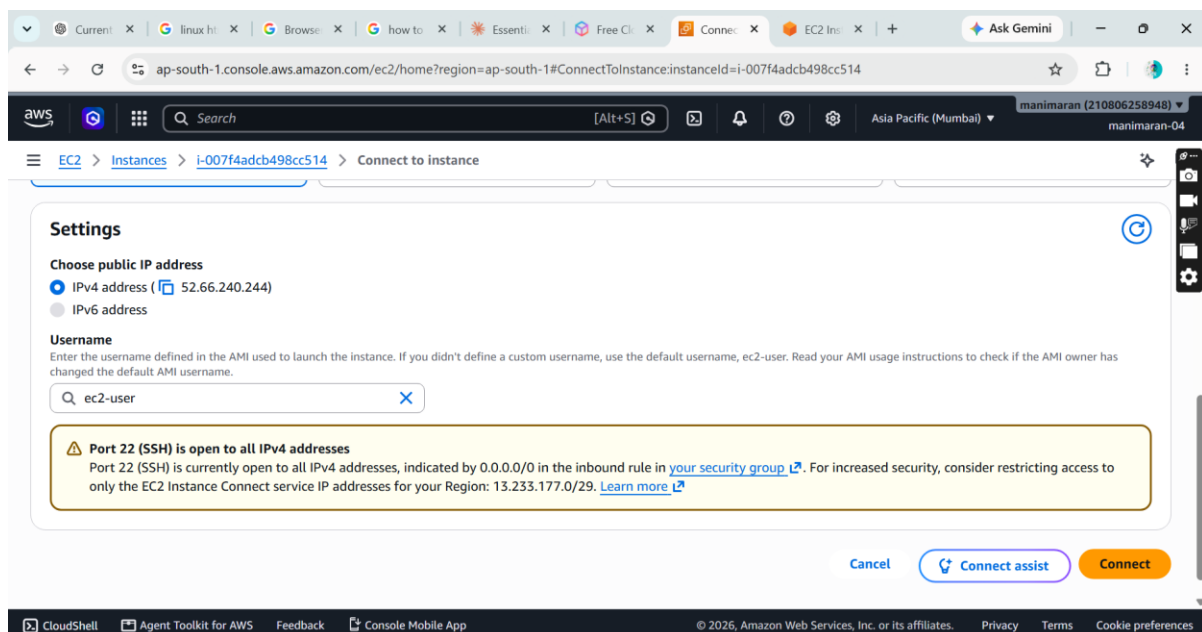
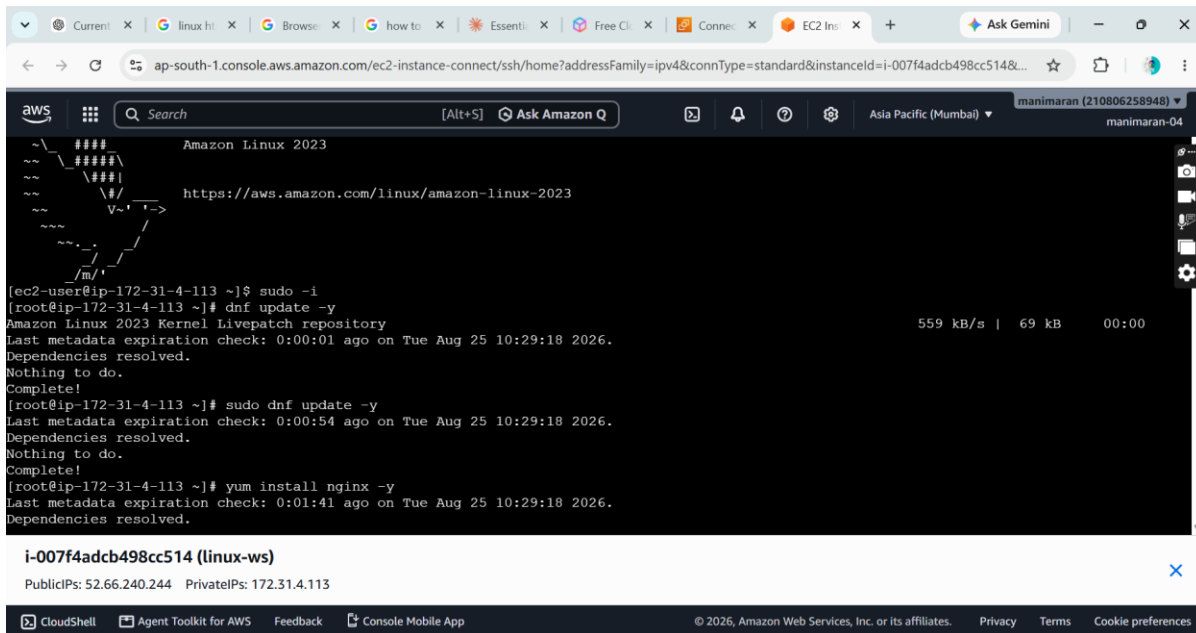


Figure 3. Evidence screenshot — Connect to the EC2 instance via EC2 Instance Connect.

Step 3 — Update the system and install Nginx

Instruction / Action: The EC2 terminal (Amazon Linux 2023) shows `sudo -i`, `dnf update -y`, `sudo dnf update -y`, and `yum install nginx -y` being run to update the system and begin the Nginx installation.



The screenshot shows a terminal window within the AWS Management Console. The terminal is connected to an Amazon Linux 2023 instance. The user runs `sudo -i` to become root. Then, `dnf update -y` is executed, showing progress for the Amazon Linux 2023 Kernel Livepatch repository. After another `dnf update -y`, the user runs `yum install nginx -y`. The terminal output shows the installation of Nginx and its dependencies. The instance ID is `i-007f4adcb498cc514` (linux-ws).

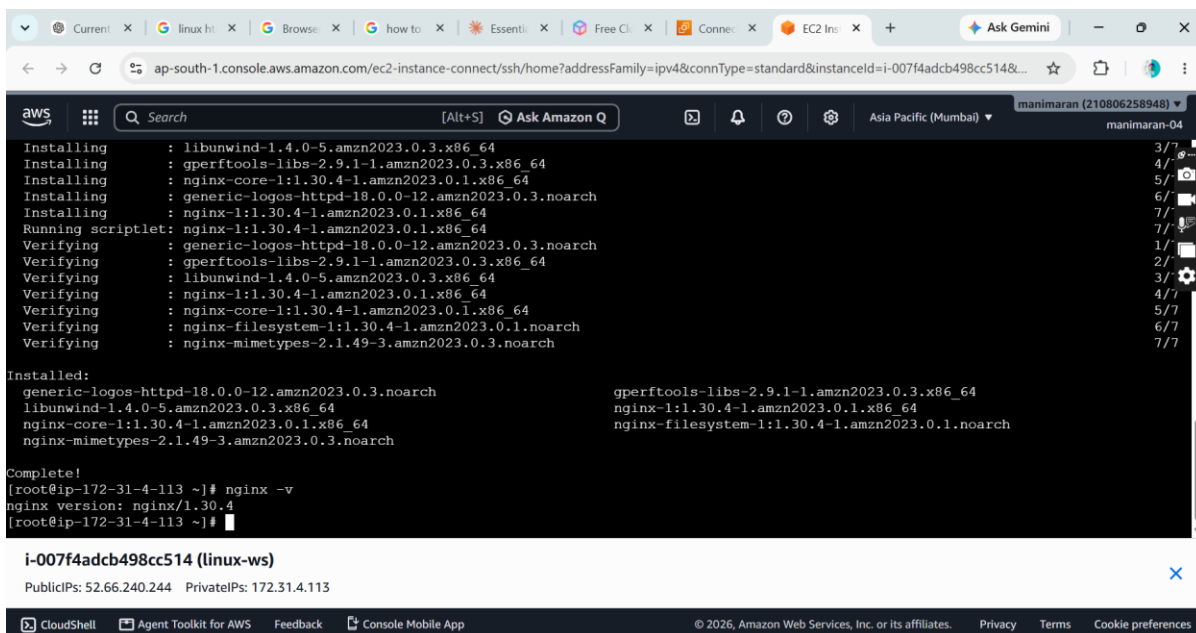
```
~# https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-4-113 ~]$ sudo -i
[root@ip-172-31-4-113 ~]# dnf update -y
Amazon Linux 2023 Kernel Livepatch repository                    559 kB/s | 69 kB    00:00
Last metadata expiration check: 0:00:01 ago on Tue Aug 25 10:29:18 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-4-113 ~]# sudo dnf update -y
Last metadata expiration check: 0:00:54 ago on Tue Aug 25 10:29:18 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-4-113 ~]# yum install nginx -y
Last metadata expiration check: 0:01:41 ago on Tue Aug 25 10:29:18 2026.
Dependencies resolved.
```

Figure 4. Evidence screenshot — Update Amazon Linux and begin installing Nginx.

Step 4 — Verify the Nginx installation

Instruction / Action: The terminal output shows the Nginx-related packages (`nginx-core`, `nginx-filessystem`, `nginx-mimetypes`, `generic-logos-httpd`, and dependencies) being installed and verified, followed by `nginx -v` confirming nginx version 1.30.4.



The screenshot shows the continuation of the terminal session. It lists the packages being installed and verified, including `libunwind`, `gperftools-libs`, `nginx-core`, `generic-logos-httpd`, `nginx`, `nginx-filessystem`, and `nginx-mimetypes`. The installation is complete, and the user runs `nginx -v`, which outputs `nginx version: nginx/1.30.4`. The instance ID is `i-007f4adcb498cc514` (linux-ws).

```
Installing      : libunwind-1.4.0-5.amzn2023.0.3.x86_64
Installing      : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
Installing      : nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64
Installing      : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
Installing      : nginx-1:1.30.4-1.amzn2023.0.1.x86_64
Running scriptlet: nginx-1:1.30.4-1.amzn2023.0.1.x86_64
Verifying       : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
Verifying       : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
Verifying       : libunwind-1.4.0-5.amzn2023.0.3.x86_64
Verifying       : nginx-1:1.30.4-1.amzn2023.0.1.x86_64
Verifying       : nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64
Verifying       : nginx-filessystem-1:1.30.4-1.amzn2023.0.1.noarch
Verifying       : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

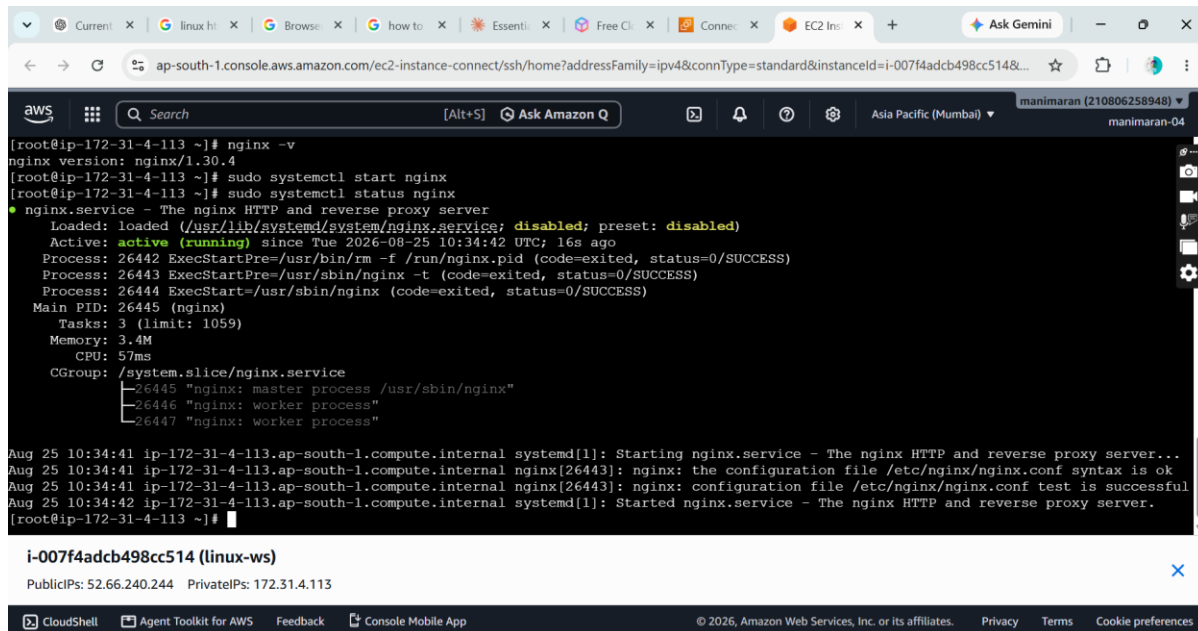
Installed:
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
libunwind-1.4.0-5.amzn2023.0.3.x86_64
nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64
nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch
gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
nginx-1:1.30.4-1.amzn2023.0.1.x86_64
nginx-filessystem-1:1.30.4-1.amzn2023.0.1.noarch

Complete!
[root@ip-172-31-4-113 ~]# nginx -v
nginx version: nginx/1.30.4
[root@ip-172-31-4-113 ~]#
```

Figure 5. Evidence screenshot — Verify Nginx installation completes and confirm the version.

Step 5 — Start Nginx and check its status

Instruction / Action: The terminal shows `nginx -v`, `sudo systemctl start nginx`, and `sudo systemctl status nginx`, confirming the service is active (running) with worker processes started.



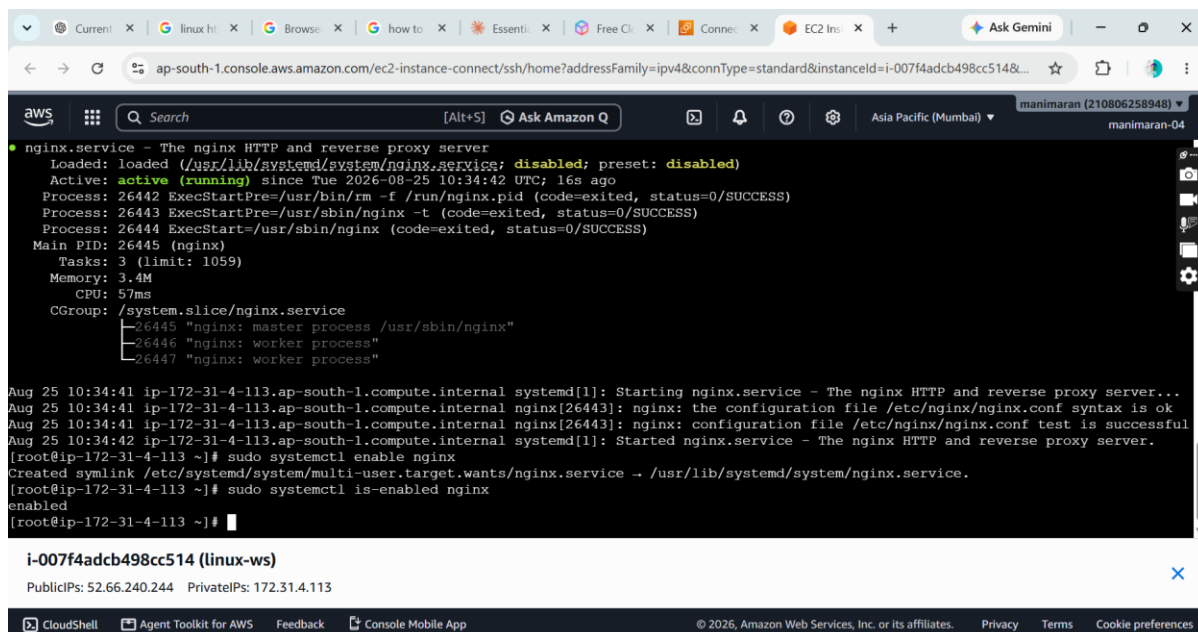
```
[root@ip-172-31-4-113 ~]# nginx -v
nginx version: nginx/1.30.4
[root@ip-172-31-4-113 ~]# sudo systemctl start nginx
[root@ip-172-31-4-113 ~]# sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-08-25 10:34:42 UTC; 16s ago
     Process: 26442 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 26443 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 26444 ExecStartPre=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
    Main PID: 26445 (nginx)
       Tasks: 3 (limit: 1059)
      Memory: 3.4M
         CPU: 57ms
    CGroup: /system.slice/nginx.service
            └─26445 "nginx: master process /usr/sbin/nginx"
              └─26446 "nginx: worker process"
                └─26447 "nginx: worker process"

Aug 25 10:34:41 ip-172-31-4-113.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Aug 25 10:34:41 ip-172-31-4-113.ap-south-1.compute.internal nginx[26443]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Aug 25 10:34:41 ip-172-31-4-113.ap-south-1.compute.internal nginx[26443]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Aug 25 10:34:42 ip-172-31-4-113.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[root@ip-172-31-4-113 ~]#
```

Figure 6. Evidence screenshot — Start the Nginx service and verify its status.

Step 6 — Enable Nginx to start on boot

Instruction / Action: The terminal shows the active Nginx service status followed by `sudo systemctl enable nginx`, which creates the systemd symlink, and `sudo systemctl is-enabled nginx`, which confirms the service is enabled.



```
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-08-25 10:34:42 UTC; 16s ago
     Process: 26442 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 26443 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 26444 ExecStartPre=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
    Main PID: 26445 (nginx)
       Tasks: 3 (limit: 1059)
      Memory: 3.4M
         CPU: 57ms
    CGroup: /system.slice/nginx.service
            └─26445 "nginx: master process /usr/sbin/nginx"
              └─26446 "nginx: worker process"
                └─26447 "nginx: worker process"

Aug 25 10:34:41 ip-172-31-4-113.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Aug 25 10:34:41 ip-172-31-4-113.ap-south-1.compute.internal nginx[26443]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Aug 25 10:34:41 ip-172-31-4-113.ap-south-1.compute.internal nginx[26443]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Aug 25 10:34:42 ip-172-31-4-113.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[root@ip-172-31-4-113 ~]# sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service -> /usr/lib/systemd/system/nginx.service.
[root@ip-172-31-4-113 ~]# sudo systemctl is-enabled nginx
enabled
[root@ip-172-31-4-113 ~]#
```

Figure 7. Evidence screenshot — Enable Nginx so it starts automatically on boot.

Step 7 — Validate the default Nginx page

Instruction / Action: The browser is pointed to the instance's public IP address (52.66.240.244) and displays the default "Welcome to nginx!" page, confirming that Nginx is reachable over HTTP.

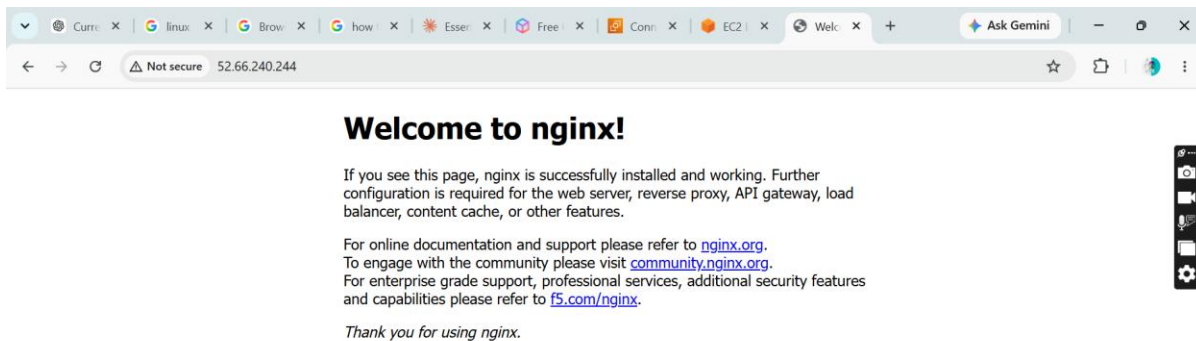


Figure 8. Evidence screenshot — Validate the default Nginx welcome page in the browser.

Step 8 — Transfer the website files using WinSCP

Instruction / Action: WinSCP is used to connect to the instance (ec2-user@52.66.240.244) over SFTP and transfer the local "mustacheenthusiast" website folder (HTML pages, CSS, JS, fonts, and images) into /usr/share/nginx/html/ on the server.

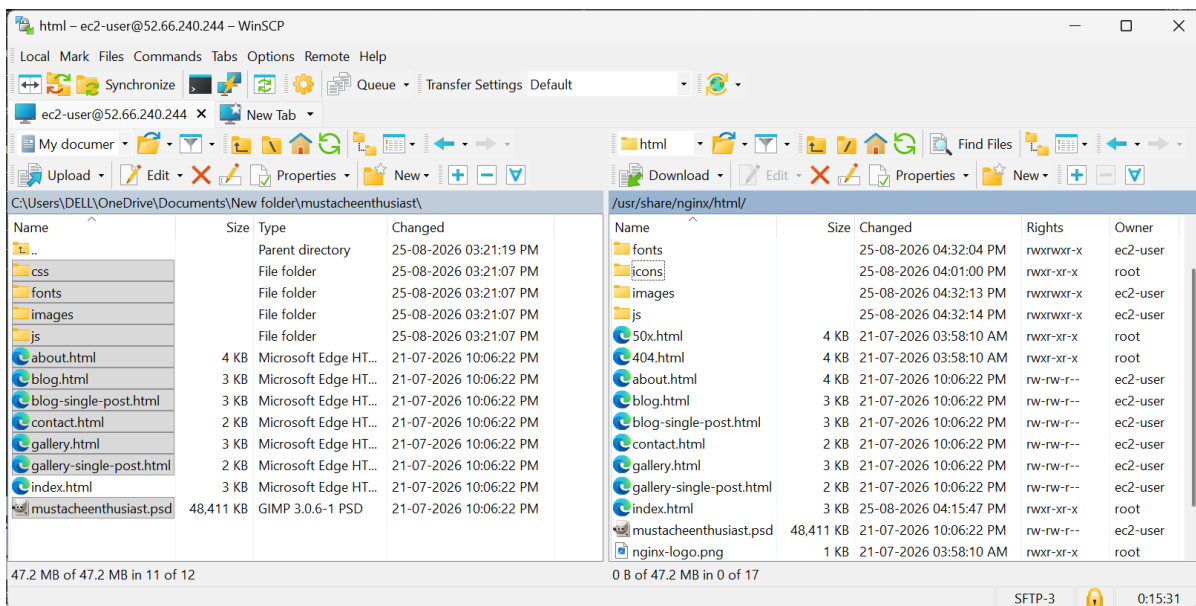


Figure 9. Evidence screenshot — Transfer the custom website files to the Nginx web root using WinSCP.

Step 9 — Validate the deployed homepage

Instruction / Action: The browser displays the deployed "Mustacchio" website homepage at the instance's public IP address, including the navigation menu, hero banner, and a "READ MORE" call-to-action button, confirming that Nginx is serving the custom content.

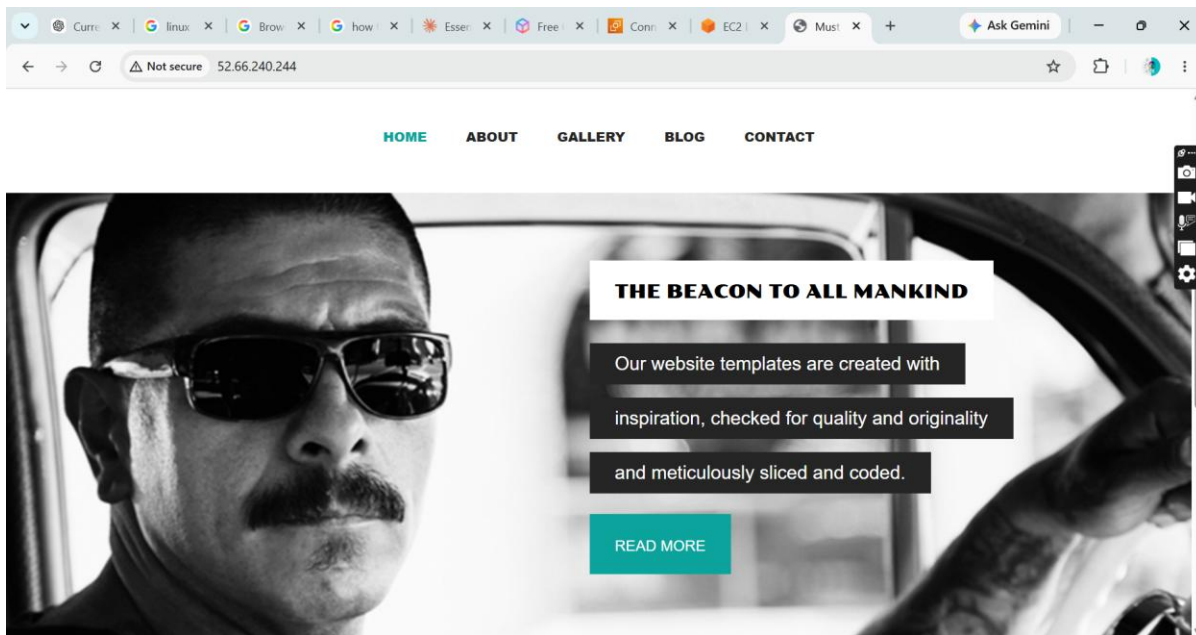


Figure 10. Evidence screenshot — Validate the deployed website homepage in the browser.

Step 10 — Correct web-root permissions and stop the service

Instruction / Action: The terminal shows the default web-root contents (404.html, 50x.html, icons, index.html, nginx-logo.png, poweredby.png) being reviewed, a stray index.html removed, the real web-root index.html edited, chmod -R 755 and chmod 777 applied to /usr/share/nginx/html to correct permissions, and the Nginx service stopped with sudo systemctl stop nginx.

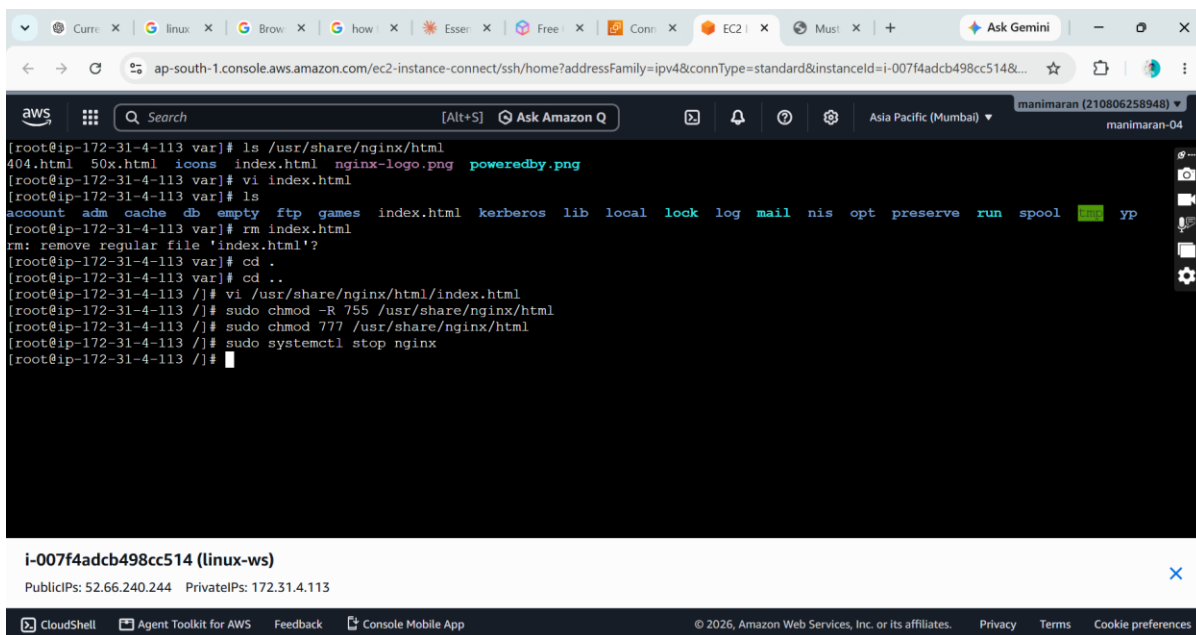


Figure 11. Evidence screenshot — Correct web-root permissions while the Nginx service is temporarily stopped.

Step 11 — Observe the connection while Nginx is stopped

Instruction / Action: With the Nginx service stopped, the browser attempts to reach the public IP address and returns "This site can't be reached — 52.66.240.244 refused to connect" (ERR_CONNECTION_REFUSED), confirming that the site is unavailable while the service is down.

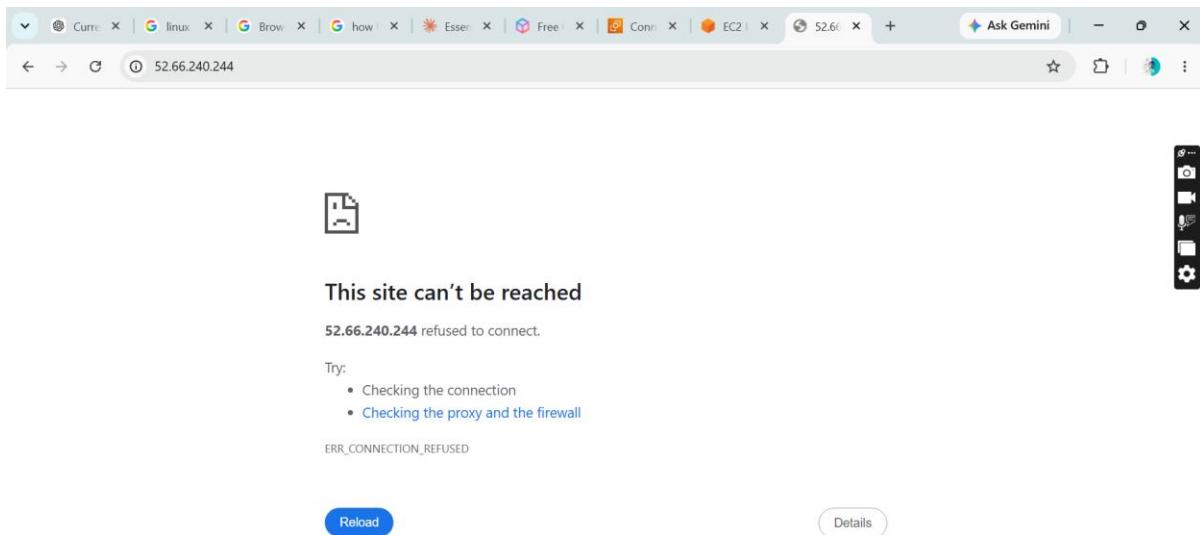


Figure 12. Evidence screenshot — Observe the connection-refused state while the Nginx service is stopped.

Step 12 — Final browser validation

Instruction / Action: After the Nginx service is restarted, the browser is pointed to `http://52.66.240.244/index.html` and successfully renders the deployed Mustacchio website, providing final confirmation that the static site is correctly hosted and served by Nginx.

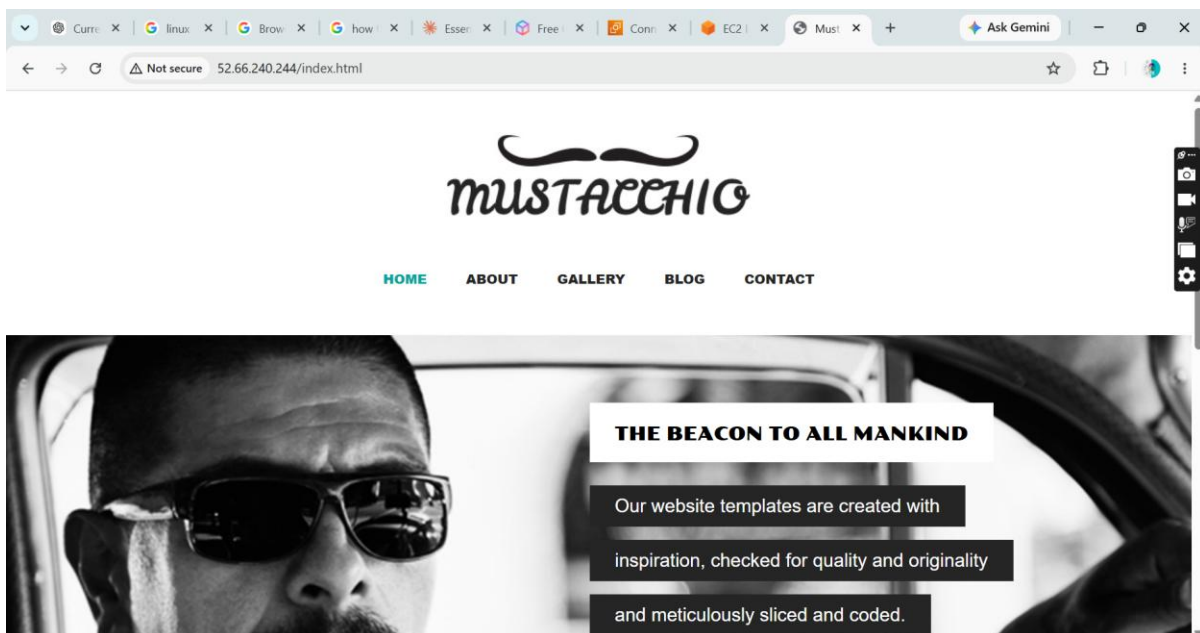


Figure 13. Evidence screenshot — Final browser validation of the deployed website.

10. Testing and Validation

Validation Area	Observed Evidence	Result
EC2 instance state	Instance "linux-ws" shown as Running with a public IP address.	Passed
Remote connectivity	EC2 Instance Connect session established using ec2-user over SSH.	Passed
Nginx installation	Package installation completed; nginx -v reports version 1.30.4.	Passed
Nginx service state	systemctl status nginx reports active (running).	Passed
Boot-time enablement	systemctl is-enabled nginx reports "enabled".	Passed
Default page validation	Default "Welcome to nginx!" page rendered in the browser.	Passed
File transfer	Website files visible in /usr/share/nginx/html/ via WinSCP.	Passed
Service-down behavior	Browser returns ERR_CONNECTION_REFUSED while Nginx is stopped.	Passed (expected)
Final website validation	Deployed Mustacchio website renders correctly at the public IP.	Passed

11. Troubleshooting and Operational Notes

- A stray index.html created in /var (outside the web root) was identified and removed before editing the correct file under /usr/share/nginx/html/.
- Mixed file ownership (root vs. ec2-user) on files transferred via WinSCP required chmod -R 755 and chmod 777 to be applied to the web-root directory so Nginx could serve the content correctly.
- Stopping Nginx during maintenance immediately made the site unreachable (ERR_CONNECTION_REFUSED); this is expected behavior and confirms the service was genuinely controlling site availability.
- If the site is unreachable after deployment, verify the Nginx service status first, then check the security group for inbound HTTP access, then verify file permissions in the web root.
- If updated files do not appear in the browser, confirm the WinSCP upload completed and clear the browser cache before retesting.

12. Security Considerations

- Restrict SSH (port 22) inbound access to specific trusted IP ranges rather than 0.0.0.0/0 where possible.
- Avoid running long-term administration as the root user; prefer sudo with a dedicated account.
- Avoid setting overly permissive file permissions (e.g., chmod 777) on production web content; scope permissions to what Nginx actually requires.
- Serve the site over HTTPS/TLS in a production scenario rather than plain HTTP.
- Keep the Amazon Linux system and Nginx package updated to receive security patches.
- Do not expose SSH credentials or key files; use EC2 Instance Connect or key-based authentication only.

13. Advantages

- Provides hands-on experience with core EC2 and Nginx administration tasks.
- Demonstrates the complete manual path from a bare Linux instance to a live, browser-accessible website.
- Low-cost and simple to set up using a single t3.micro EC2 instance.
- Establishes the operational foundation needed before introducing automation (as in a Level 3 CI/CD pipeline).

14. Limitations

- Deployment is entirely manual; there is no automated build, test, or release pipeline.
- The site is served over plain HTTP without TLS encryption.
- There is no load balancing, auto scaling, or redundancy — a single EC2 instance is a single point of failure.
- File transfer and permission management are performed manually and are prone to human error, as seen in the permission-correction step.

15. Future Enhancements

- Introduce a CI/CD pipeline (e.g., GitHub + Jenkins) to automate deployment, as in a Level 3 implementation.
- Add HTTPS/TLS using a managed certificate.
- Move to an Auto Scaling Group behind an Application Load Balancer for high availability.
- Add CloudWatch monitoring and alerting for the EC2 instance and Nginx service.
- Standardize file permissions and ownership using a deployment script instead of manual chmod commands.
- Manage the infrastructure using Infrastructure as Code (e.g., Terraform or CloudFormation).

16. Key Lessons Learned

- A newly installed Nginx service must be both started and enabled to be available immediately and after reboot.
- File ownership and permissions matter: files uploaded via SFTP can end up with inconsistent ownership that blocks the web server from serving them correctly.
- Stopping the web server immediately and directly affects site availability, which is useful for confirming that Nginx — not networking — controls reachability.
- Validating at both the service level (systemctl status) and the application level (browser access) gives more confidence than either check alone.

17. Interview / Viva Explanation

"I deployed a static portfolio website on an AWS EC2 instance running Amazon Linux 2023, using Nginx as the web server. After launching and connecting to the instance, I updated the system and installed Nginx, then started and enabled the service so it would persist across reboots. I validated the default Nginx page in the browser, then used WinSCP to transfer my website's HTML, CSS, JS, and asset files into the Nginx web root. I corrected file permission issues that resulted from the transfer, restarted Nginx, and confirmed that stopping the service made the site unreachable, which validated that Nginx was correctly controlling access. Finally, I validated the fully deployed website in the browser using the instance's public IP address."

18. Conclusion

This Level 2 project demonstrates a practical, manual static-website deployment on AWS EC2 using Nginx. The evidence documents instance verification, remote connectivity, Nginx installation and service management, file transfer via WinSCP, permission troubleshooting, and both service-level and browser-level validation of the final deployed website.

The demonstrated delivery path is: User Browser → AWS Internet Gateway → EC2 Instance (Amazon Linux) → Nginx → Website Files → User Browser.

Appendix A — Implementation Evidence Inventory

The complete submitted evidence set for this Level 2 project contains 12 individual implementation screenshots, together with 1 separate system architecture diagram. The screenshots are retained in chronological order in Section 9, and the architecture diagram is included in Section 6.

Evidence Group	Count	Description
Instance & connectivity	2 screenshots	EC2 instance verification and EC2 Instance Connect setup.
Nginx installation & service	4 screenshots	System update, Nginx install, service start/status, and boot-time enablement.
Deployment & browser checks	6 screenshots	Default Nginx page, WinSCP file transfer, homepage validation, permission correction, connection-refused state, and final validation.
System architecture	1 diagram	Supplied Level 2 EC2/Nginx static hosting architecture.

End of Level 2 Project Documentation